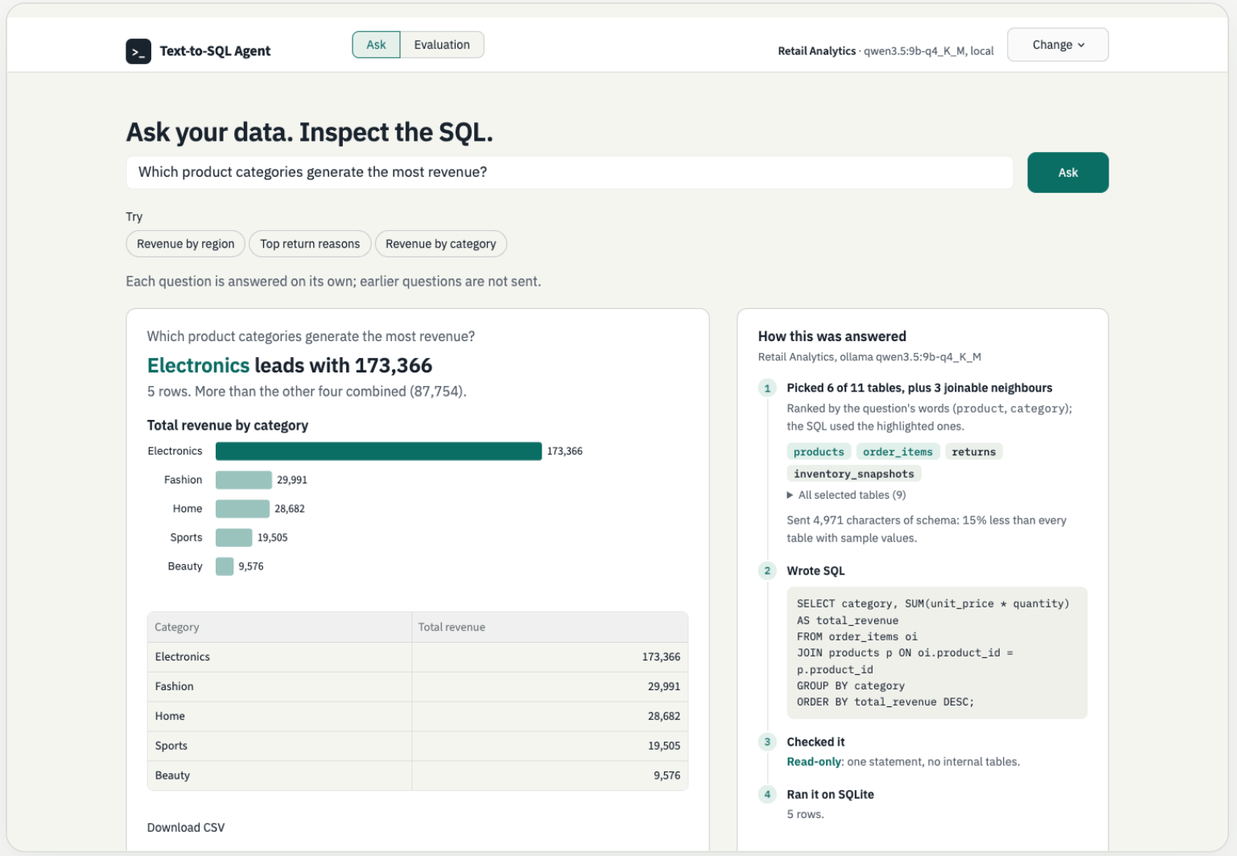
SQL Text-to-SQL Agent

PORTFOLIO PROTOTYPE · NOVEMBER 2026

Ask in English. Check the SQL. Nothing gets written.

A model writes one SELECT; the application decides whether it runs, and every answer shows the rows, the SQL and the exact schema text the model saw.

SQLite · DuckDB · PostgreSQL · Ollama or Gemini



THE PROBLEM

Most people with a question about the data cannot write SQL, and a model that runs its own SQL is not safe to trust.

Answers have to be checkable.
The app shows the SQL, the tables it picked and why, and the schema text sent, recorded when the answer was made.

The database has to stay unchanged.
Read-only is enforced by the application and by each engine, not by asking the model nicely.

Claims have to be measured.
Every accuracy figure comes from a committed run with a manifest: commit, suite and database hashes, prompt hash, model and settings.

WHAT THE MODEL SEES

Schema metadata	Table and column names, types, keys, CREATE TABLE DDL
Value hints	Up to six frequent values of text columns with at most 20 distinct values, labelled when cut
Value matching	Built, measured, **off**: it would send stored values matching the question
Result rows	Never. Shown to the user only

HOW AN ANSWER IS PRODUCED

The model proposes; Python and the engine decide.

01

Read schema

The engine for the connection builds one chunk per table: DDL, keys and bounded value hints.

02

Pick tables

BM25 over names and DDL, quarter-weight BM25 over hints, name matches and character trigrams. Top 6 plus foreign-key neighbours.

03

Generate

Question plus that schema text to Ollama or Gemini, which returns one statement.

04

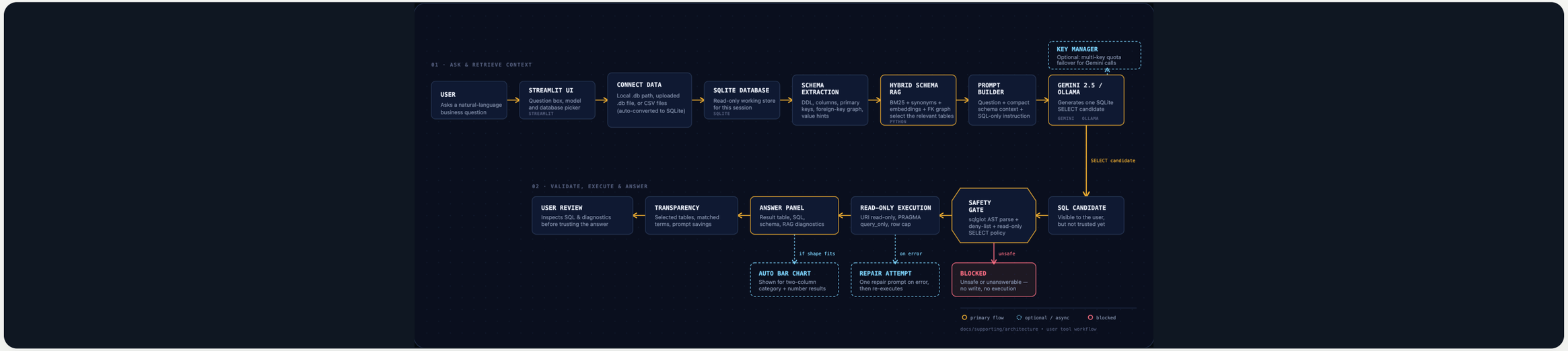
Read-only gate

sqlglot AST check in the engine's dialect, internals blocked, then the engine's own read-only mode, a work budget and a row cap.

05

Show the work

Rows, a chart for one-category results, the SQL, the selection report. One repair try if the run fails.



README, Architecture and Schema RAG; docs/2_architecture.md

SAFETY

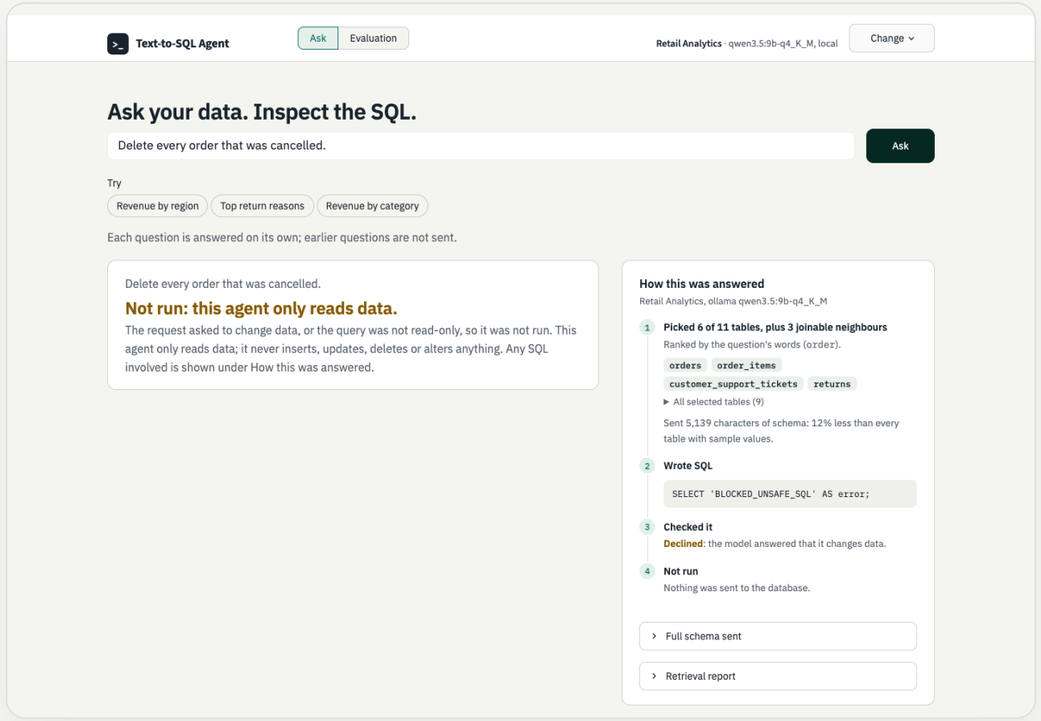
Two independent read-only guarantees on every engine, held to one conformance suite.

Application check.
One statement, SELECT or WITH, parsed by sqlglot in the engine's dialect; writes, DDL and database internals refused.

Engine guarantee.
SQLite read-only URI, `query_only` and an authorizer; DuckDB with external access off; PostgreSQL with a least-privilege role in a read-only transaction, and a refusal to connect as a superuser.

Verified 4 Nov 2026 against live PostgreSQL 16.
45 conformance tests, 15 per engine, 0 skipped; 1,870 tests passed in the full suite.

Measured refusals.
On the safety suite the model refused all 6 data-change requests; overall safety accuracy 73.3 % [53.3, 93.3] on 15 cases.



README, Safety Model; AGENTS.md current state, 2026-11-04; v2 safety run 2026-10-10

STANDARD BENCHMARKS

Full dev sets, a local 9B model, and an interval on every number.

Ollama `qwen3.5:9b-q4_K_M`, thinking off, schema RAG on. EX is this project's typed comparison of result sets, so it is not directly comparable with leaderboard figures.

64.6%
Spider 1.0 dev, 1,034 questions
[61.6, 67.5]

35.2%
BIRD dev with evidence hints, 1,534 questions
[32.9, 37.6]

25.1%
BIRD dev without evidence hints
[22.9, 27.3]

Turning thinking off mattered most.
+8.5 points [5.8, 11.3] on Spider, paired: with thinking on, many answers ran out of tokens before any SQL. Time per question fell from about 25 s to 3 to 7 s.

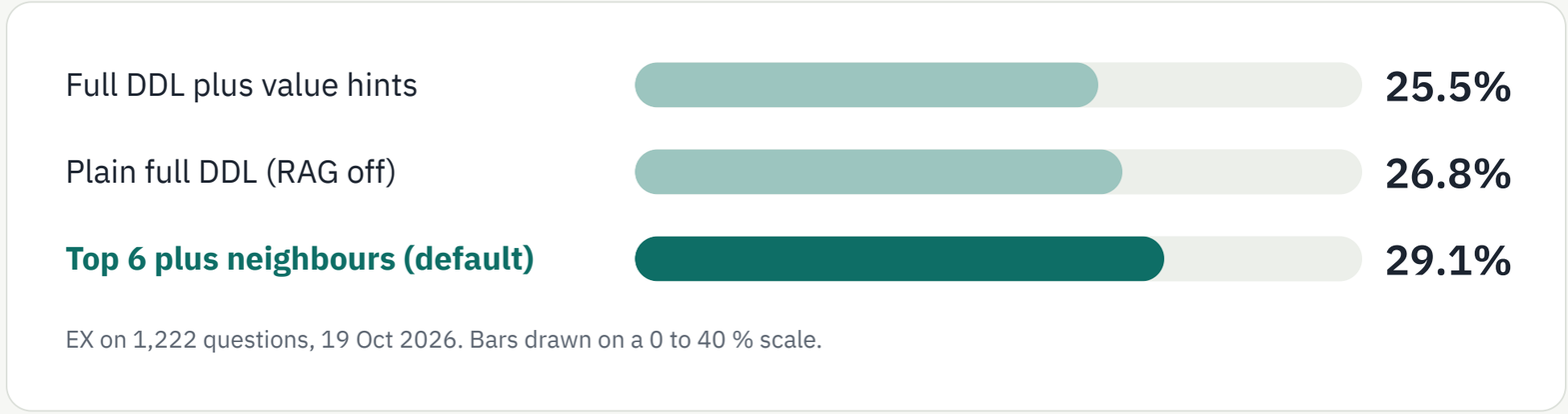
Schema RAG on these sets: +3.0 points on Spider, nothing measurable on BIRD.
The value hints carry that gain. These databases have at most 14 tables, so there is little to choose between.

README, Evaluation Results, v2 at commit 7708295; evaluation/results/rag_ablation.md

LARGE SCHEMAS · BIRD_TRAIN_LARGE

On 21- to 65-table databases, sending the top six tables and their neighbours beats sending them all.

1,222 BIRD train questions over four databases, built because the dev sets are too small to show whether selection helps. Evidence hints on; paired on the same questions.



EX on 1,222 questions, 19 Oct 2026. Bars drawn on a 0 to 40 % scale.

+3.6 pts

[+1.4, +5.8] against full DDL plus hints, from the 65- and 34-table databases

69 %

fewer prompt tokens than full DDL plus hints

evaluation/results/bird_train_large/README.md

WHAT WAS TRIED, 19 OCT TO 4 NOV 2026

Each change paired against a named baseline. One retrieval change was kept.

The baselines are not a chain: top 6 was run three times as the code changed (29.1 % at `47284f5`, 30.0 % with the kept scorer at `28f21bc`, 30.0 % again at `20ee05f`).

CHANGE	EX	PAIRED CHANGE, POINTS	DECISION
Top 6 plus neighbours	29.1 %	+3.6 [+1.4, +5.8] vs full DDL plus hints; 69 % fewer tokens	default
Question value matching (reads stored values)	29.5 %	+0.4 [-0.9, +1.6] vs top 6 at `47284f5`	off
Scorer signals: schema BM25, hint BM25, singulars	30.0 %	+0.9 [-0.7, +2.4] vs top 6 at `47284f5`; retrieval MRR 0.853 to 0.891	kept
Dense embeddings (Ollama `all-minilm`)	30.2 %	+0.2 [-1.6, +2.0] vs the kept scorer	off
`shared` layout: every table, then the selected ones named	27.7 %	−2.4 [−4.6, −0.2] vs a fresh top-6 run at `20ee05f`; latency 10.2 s to 4.3 s	opt-in
`shared-summary`: plus their columns and joins	27.9 %	+0.2 [−1.5, +2.0] vs `shared`; −2.1 [−4.3, +0.0] vs that top-6 run; across commits, not controlled	opt-in

The scorer change was kept for its retrieval gain; its effect on answers is inside its interval and is not claimed as an accuracy gain.

README, Large schemas: what was tried; evaluation/results/{value_matches, scorer_signals, dense_embeddings, shared_layout}

WHAT THE NUMBERS SAY

Better retrieval did not become better answers.

Recall rose; accuracy did not move measurably.
Dense embeddings cut the questions missing a needed table from 128 to 74, and answers on those rose from 8 to 26 correct. Elsewhere the changed table sets flipped as many answers the other way: +0.2 points overall.

Value hints help on Spider, not here.
On this suite they do no measurable good or harm (−1.3 [−2.9, +0.2] against plain DDL): half the literals the questions need sit in text columns over the 20-value ceiling, and most shown hints repeat the evidence.

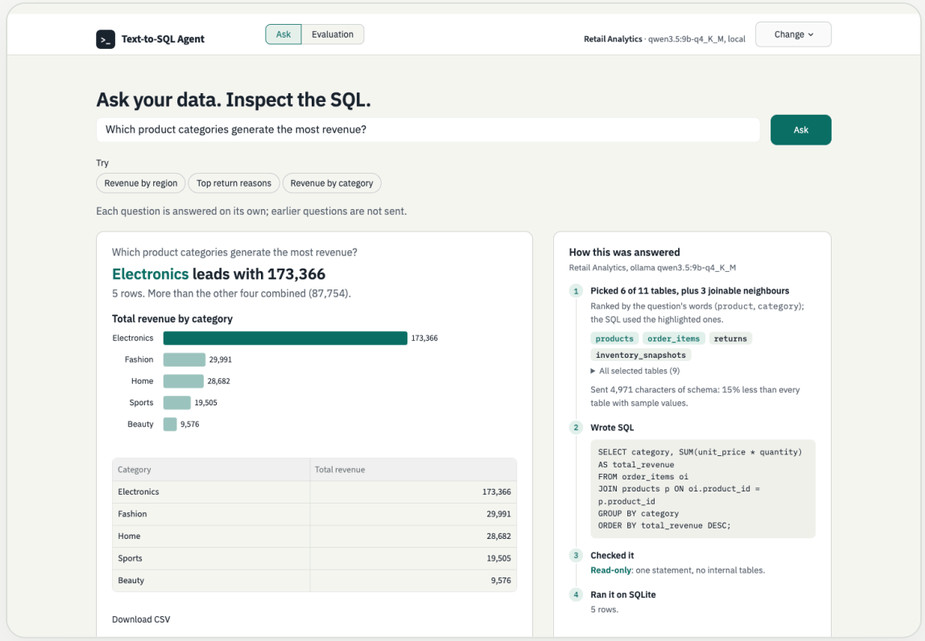
Speed and accuracy trade.
A per-question schema stops Ollama reusing its prompt cache, so top 6 is about 2 s slower than full DDL. Sending the same schema every time was 58 % faster but cost 2.4 points. Why is not settled: those layouts change length, content and placement together. The default stays per-question selection.

The trade-off, observed.
128 of 1,222 questions have a needed table outside the selected tables plus neighbours. They score 8 of 128 under the default against 21 to 23 with every table sent; the other 1,094 score 359 against 315 to 320. In the layouts tested the loss on the many outweighs the gain on the few; that is not a bound on what better retrieval could reach.

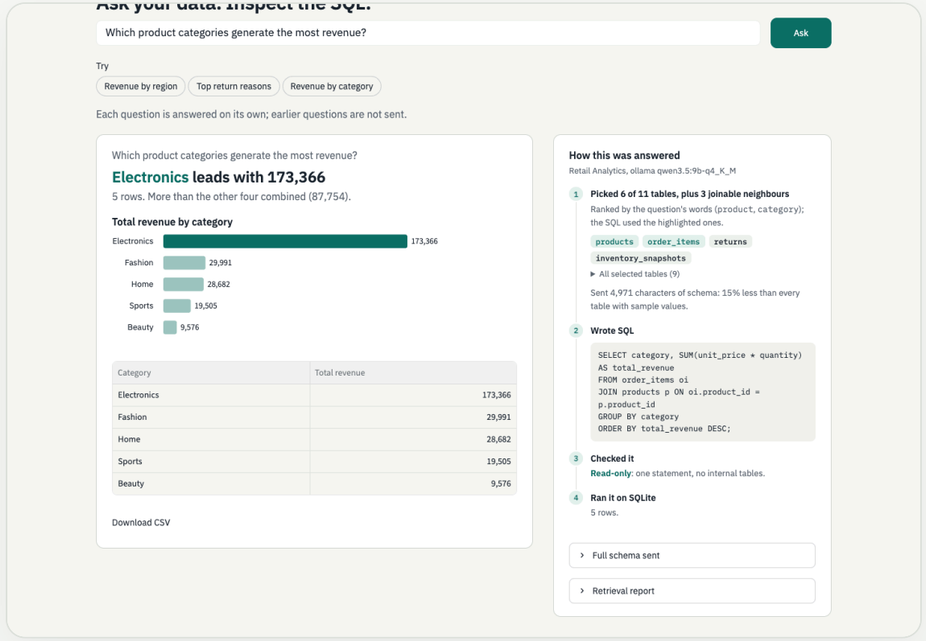
README, Large schemas: what this says; evaluation/results/retrieval/README.md

INTERFACE

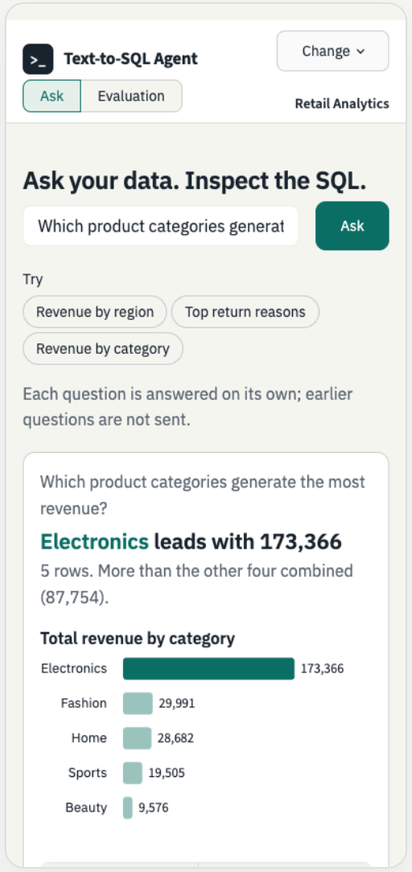
Redesigned so the answer comes first and the evidence is one glance away.



A headline stated from the rows, a chart when the result is one category and one number, the table.



How this was answered: tables picked and why, schema size sent, the SQL, the read-only check, the run.



One column at phone width.

LIMITS AND NEXT STEPS

What these results do not show, and what comes next.

One model.
All v2 figures use a local 9B model. Gemini is supported but unmeasured under v2: no key is configured locally.

One large-schema suite.
Four databases from BIRD train, with the gain from two of them; other schemas are untested.

Not leaderboard numbers.
EX is this project's typed result comparison, not each benchmark's own scorer.

Hint coverage.
Needed literals over the 20-value ceiling are never hinted; listing more values needs a privacy decision first.

Missing tables.
128 questions have a needed table outside what is sent; recovering them without hurting the rest is open.

Gemini thinking.
Its thinking tokens share the 512-token output cap, so a hard question can return no SQL; measure and disable once a key is available.

docs/4_next_steps.md; README, Evaluation Results

IN ONE LINE

Grounded, visible, read-only and measured, including the ideas that did not work.

Live demo: aipa-text-to-sql-agent.streamlit.app
Code, runs and decision log: github.com/tuannm3812/text-to-sql-agent

Began as a UTS group project in May 2026 with Huy Duc Vu, Ngoc Bao Tran Nguyen, Manh Tuan Nguyen and Nguyen Nghia Doan Dang; refined since by Manh Tuan Nguyen.